

# Q1. You are given a /20 VPC CIDR.

Design exact subnet CIDRs for:

- 2 public subnets
- 2 private subnets
- Each subnet must support at least 200 usable IPs

Explain why your CIDR choices work, and show AWS console proof that IP exhaustion does not occur.

Ans :

The screenshot shows the AWS Management Console for a VPC named 'demo-vpc' with ID 'vpc-0b4d7faaf470b481e'. The VPC is in the 'Available' state and has an IPv4 CIDR of 192.168.0.0/20. The 'Resource map' tab is selected, showing the VPC's internal structure. It includes four subnets: 'eu-north-1a' (containing 'pri-sub1' and 'pub-sub1') and 'eu-north-1b' (containing 'pri-sub2' and 'pub-sub2'). A single route table, 'rtb-02b5dab6cf8700550', is shown connected to the 'my-internet-gateway'.

The screenshot shows the details of a subnet named 'pub-sub1' with ID 'subnet-0b521297c56bde054'. The subnet is in the 'Available' state and has an IPv4 CIDR of 192.168.1.0/24. It is located in the 'eu-north-1a' availability zone. The 'Available IPv4 addresses' section shows 251 available addresses. The subnet is connected to the 'demo-vpc' and has a route table 'rtb-02b5dab6cf8700550' associated with it. The 'Flow logs' section shows no flow logs found.

I have designed subnet using CIDR : 192.168.0.0/24 which gives 256 total IPs , after AWS reserves 5IPs there are still 251 usable IP addresses which makes my choice suitable and according to the need.

I have designed two public IP subnets : 192.168.0.0/24 and 192.168.1.0/24 and two private subnets : 192.168.2.0/24 and 192.168.3.0/24. My public subnets are connected to my internet gateway.

The AWS console shows that there are 251 available IP addresses and IP exhaustion will not occur until the requirement is below 250.

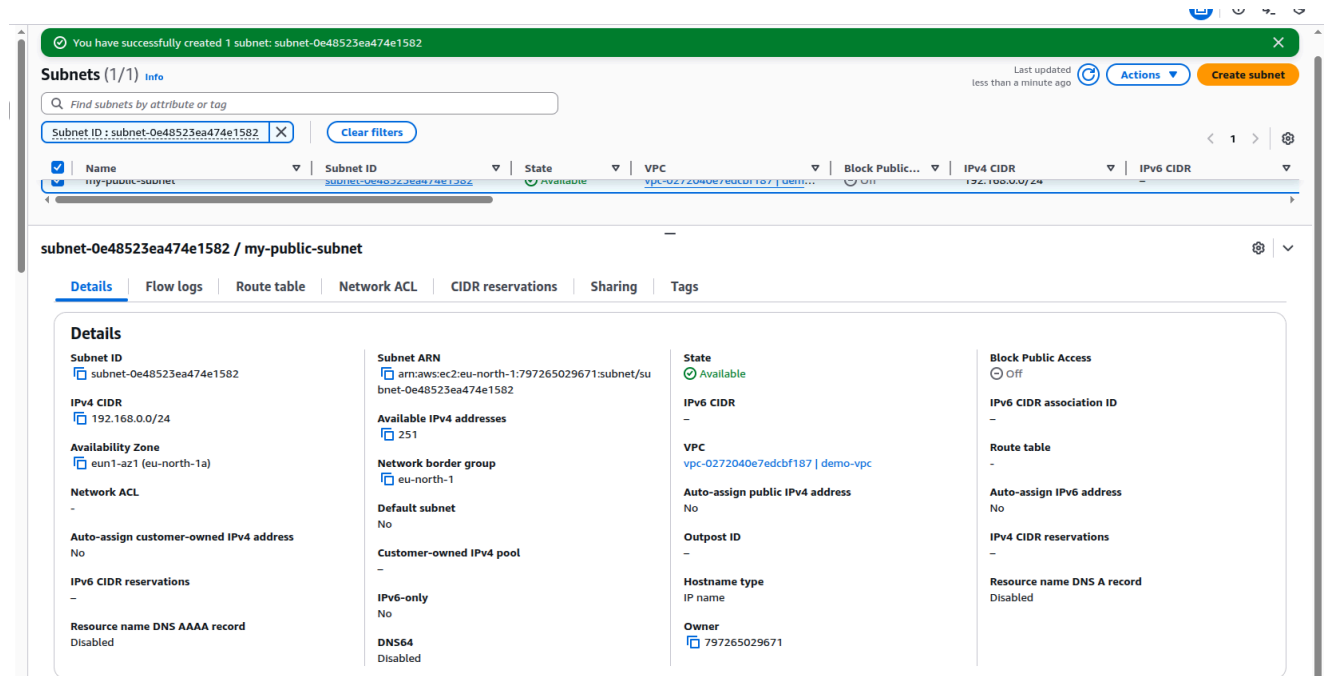
## Q2. Create a subnet you claim is public.

Prove it by showing:

- A. Route table entry
- B. Internet Gateway attachment
- C. EC2 reachability test from the internet

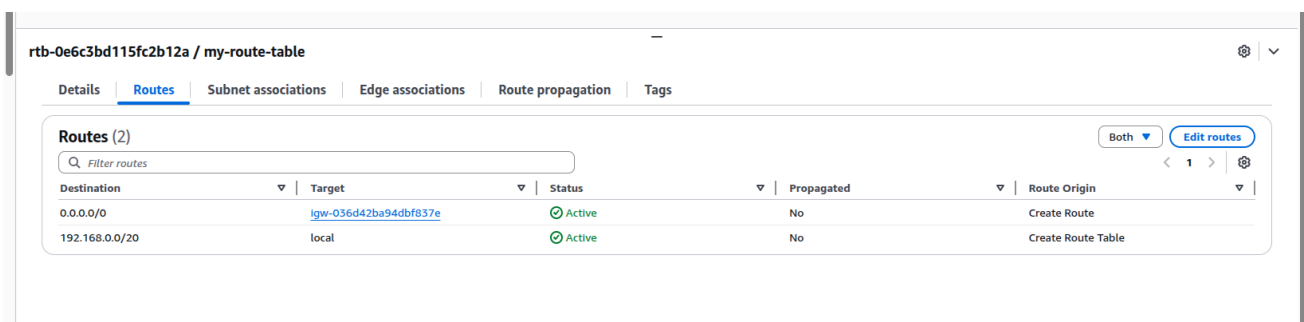
Explain what single change would immediately make it private again.

Ans :



My Public subnet is created using the CIDR : 192.168.0.0/24 . My network CIDR is : 192.168.0.0/20.

A .



My route table has a route 0.0.0.0/0 from my internet gateway , making my subnet as public and

accessible from the internet. This will allow direct inbound and outbound internet traffic.

B. Internet gateway is associated with the my public subnet . The internet gateway I created is named my-internet-gateway and it is attached to my vpc and the subnet making it public.

Internet gateway igw-036d42ba94dbf837e successfully attached to vpc-0272040e7edcbf187

## igw-036d42ba94dbf837e / my-internet-gateway

Actions

**Details** info

**Internet gateway ID**  
 igw-036d42ba94dbf837e

**State**  
 Attached

**VPC ID**  
 vpc-0272040e7edcbf187 | demo-vpc

**Owner**  
 797265029671

**Tags (1)**

Search tags

| Key  | Value               |
|------|---------------------|
| Name | my-internet-gateway |

Manage tags

C.

[Alt+S]

Europe (Stockholm) | siriusdevops (7972-6502-9671) | siriusdevops

## Instances (1/1) info

Find Instance by attribute or tag (case-sensitive) | All states

Instance state = running | Clear filters

| Name                           | Instance ID | Instance state | Instance type | Status check | Alarm status | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elastic II |
|--------------------------------|-------------|----------------|---------------|--------------|--------------|-------------------|-----------------|-----------------|------------|
| i-0880244833b612f46 (demo-ec2) |             |                |               |              |              |                   |                 |                 |            |

**Instance summary** info

**Instance ID**  
 i-0880244833b612f46

**IPv6 address**  
 -

**Hostname type**  
 IP name: ip-192-168-0-211.eu-north-1.compute.internal

**Answer private resource DNS name**  
 -

**Auto-assigned IP address**  
 51.20.121.226 [Public IP]

**IAM Role**  
 -

**IMDSv2**  
 Required

**Operator**  
 -

**Public IPv4 address**  
 51.20.121.226 | open address

**Instance state**  
 Running

**Private IP DNS name (IPv4 only)**  
 ip-192-168-0-211.eu-north-1.compute.internal

**Instance type**  
 t3.micro

**VPC ID**  
 vpc-0272040e7edcbf187 (demo-vpc)

**Subnet ID**  
 subnet-0e48523ea474e1582 (my-public-subnet)

**Instance ARN**  
 arn:aws:ec2:eu-north-1:797265029671:instance/i-0880244833b612f46

**Private IPv4 addresses**  
 192.168.0.211
 
**Public DNS**  
 -
 
**Elastic IP addresses**  
 -
 
**AWS Compute Optimizer finding**  
 Opt-in to AWS Compute Optimizer for recommendations. | Learn more
 
**Auto Scaling Group name**  
 -
 
**Managed**  
 false
 

I have create my EC2 instance and configured it for my VPC named demo-vpc and now I use my terminal to check the reachability.

```
piyush@CK-piyush: ~  
In piyush@CK-piyush:~$ curl http://51.20.121.226  
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/st  
rict.dtd">  
<html>  
<head>  
<title>It works! Apache httpd</title>  
</head>  
<body>  
<p>It works!</p>  
</body>  
</html>  
piyush@CK-piyush:~$
```

I used the linux command `curl` followed by my public ip address and the instance responded with a html page this shows connection is succesfull.

Removing the route table entry `0.0.0.0/0 - igw-my-internet-gateway` will make the subnet as private no matter what the name shows.

Updated routes for rtb-0e6c3bd115fc2b12a / my-route-table successfully

rtb-0e6c3bd115fc2b12a / my-route-table

| Details        |                                  | Main     | Explicit subnet associations                | Edge associations |
|----------------|----------------------------------|----------|---------------------------------------------|-------------------|
| Route table ID | rtb-0e6c3bd115fc2b12a            | No       | subnet-0e48523ea474e1582 / my-public-subnet | -                 |
| VPC            | vpc-0272040e7edcbf187   demo-vpc | Owner ID | 797265029671                                |                   |

Routes (1)

| Destination    | Target | Status | Propagated | Route Origin       |
|----------------|--------|--------|------------|--------------------|
| 192.168.0.0/20 | local  | Active | No         | Create Route Table |

Now my subnet is private and will not be able to connect.

```

^C
piyush@CK-piyush:~$ curl -v http://51.20.121.226
* Trying 51.20.121.226:80...
* connect to 51.20.121.226 port 80 from 192.168.1.84 port 55218 failed: Connection timed out
* Failed to connect to 51.20.121.226 port 80 after 132904 ms: Couldn't connect to server
* Closing connection
curl: (28) Failed to connect to 51.20.121.226 port 80 after 132904 ms: Couldn't connect to server
piyush@CK-piyush:~$

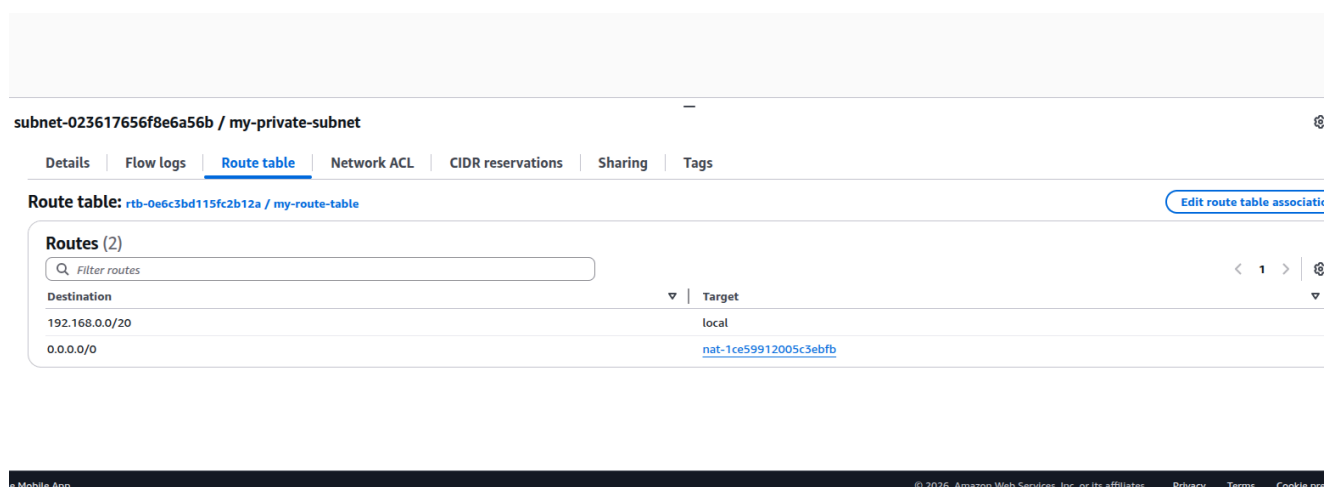
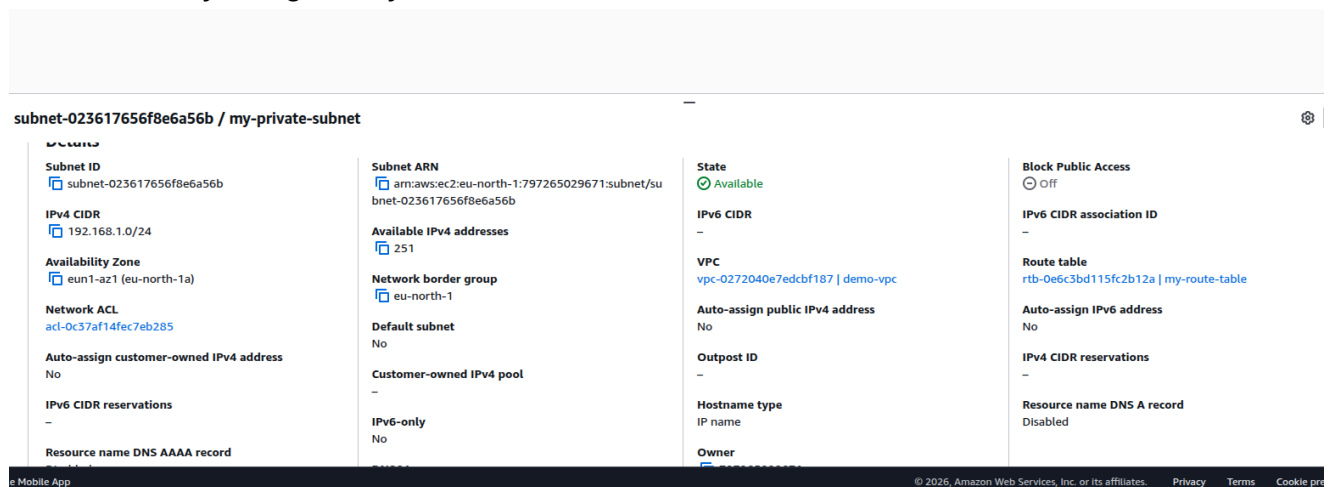
```

### Q3. Create a private subnet using a NAT Gateway for outbound access.

Then intentionally break internet access for private EC2 without deleting the NAT Gateway.  
Document:

- What you changed
- Why traffic fails
- How you verified failure

**Ans :** My Private subnet created using the CIDR : 192.168.1.0 and attached a NAT gateway to it named my-nat-gateway.



My private subnet is connected to my NAT gateway to handle outbound traffic.

To intentionally break internet access for private EC2 instance without deleting the NAT Gateway .

A. I changed the route table entry for the private subnet network and the associated NAT Gateway .

Updated routes for rtb-0e6c3bd115fc2b12a / my-route-table successfully  
Details

rtb-0e6c3bd115fc2b12a / my-route-table

**Details** info

|                                         |                          |                                                                              |                        |
|-----------------------------------------|--------------------------|------------------------------------------------------------------------------|------------------------|
| Route table ID<br>rtb-0e6c3bd115fc2b12a | Main<br>No               | Explicit subnet associations<br>subnet-023617656f8e6a56b / my-private-subnet | Edge associations<br>- |
| VPC<br>vpc-0272040e7edcbf187   demo-vpc | Owner ID<br>797265029671 |                                                                              |                        |

**Routes** | Subnet associations | Edge associations | Route propagation | Tags

**Routes (1)**

Filter routes

| Destination    | Target | Status | Propagated | Route Origin       |
|----------------|--------|--------|------------|--------------------|
| 192.168.0.0/20 | local  | Active | No         | Create Route Table |

B. The traffic fails because there is no entry for the inbound rule in the route table which drops the packet silently as there is no matching route for 0.0.0.0/0.

C. Since the NAT gateway is not deleted and it exists , so I verified the route table and it did not have any default route which was the problem. The subnet now has not path for internal bound traffic .

## Q4. You created a VPC with a /24 CIDR and deployed: 2 public subnets and 2 private subnets .

Six months later, the private subnets are running out of IP addresses.

Your task:

- A. Expand the network capacity without deleting the VPC
- B. Without changing the existing VPC primary CIDR
- C. Without downtime to running EC2 instances

**Ans :** I created a VPC with CIDR /24 and deployed 2 public and 2 private subnets each in one AZ .

my-vpc vpc-06b6c7efd41dad953 Available - - Off 10.0.0.0/24 -

vpc-06b6c7efd41dad953 / my-vpc

**Resource map** info

**VPC**  
Your AWS virtual network  
my-vpc

**Subnets (4)**  
Subnets within this VPC

- eu-north-1a
  - public-subnet01
  - private-subnet01
- eu-north-1b
  - public-subnet02
  - private-subnet02

**Route tables (1)**  
Route network traffic to resources  
rtb-0d3935c9bf7cb932b

**Network Connections (0)**  
Connections to other networks

The Subnets are create with CIDR /26 which offers 59 usable IPs which may result in IP exhaustion in future.

|                                     |                  |                          |           |                                 |     |              |   |
|-------------------------------------|------------------|--------------------------|-----------|---------------------------------|-----|--------------|---|
| <input checked="" type="checkbox"/> | public-subnet01  | subnet-070910d92edb2cd26 | Available | vpc-06b6c7efd41dad953   my-v... | Off | 10.0.0.0/26  | - |
| <input type="checkbox"/>            | private-subnet01 | subnet-091182fd68ba43364 | Available | vpc-06b6c7efd41dad953   my-v... | Off | 10.0.128/26  | - |
| <input type="checkbox"/>            | public-subnet02  | subnet-026f3fdb4055abec  | Available | vpc-06b6c7efd41dad953   my-v... | Off | 10.0.0.64/26 | - |
| <input type="checkbox"/>            | private-subnet02 | subnet-09489531b6e34e8e3 | Available | vpc-06b6c7efd41dad953   my-v... | Off | 10.0.192/26  | - |

subnet-070910d92edb2cd26 / public-subnet01

**Details**

Subnet ID  
subnet-070910d92edb2cd26

IPv4 CIDR  
10.0.0.0/26

Availability Zone  
eu-north-1 (eu-north-1a)

Network ACL  
-

Auto-assign customer-owned IPv4 address  
No

IPv6 CIDR reservations  
-

Subnet ARN  
arn:aws:ec2:eu-north-1:797265029671:subnet/subnet-070910d92edb2cd26

Available IPv4 addresses  
59

Network border group  
eu-north-1

Default subnet  
No

Customer-owned IPv4 pool  
-

State  
Available

IPv6 CIDR  
-

VPC  
vpc-06b6c7efd41dad953 | my-vpc

Auto-assign public IPv4 address  
No

Outpost ID  
-

Hostname type  
-

Block Public Access  
Off

IPv6 CIDR association ID  
-

Route table  
-

Auto-assign IPv6 address  
No

IPv4 CIDR reservations  
-

Resource name DNS A record  
-

To expand the network capacity without deleting the VPC and changing the existing VPC primary CIDR by adding a Secondary CIDR block . By selecting the VPC and then going to Edit CIDRs , I configure and added a Secondary CIDR group to solve the problem of IP exhaustion .

**Edit CIDRs**
Add or remove CIDR blocks for your VPC.

**IPv4 CIDRs**

| CIDR        | Status     |        |
|-------------|------------|--------|
| 10.0.0.0/24 | Associated | Remove |
| 10.1.0.0/24 | Associated | Remove |

Add new IPv4 CIDR

**IPv6 CIDRs**

| CIDR (Network border group)                            | Pool | Address attribute | Status |
|--------------------------------------------------------|------|-------------------|--------|
| You have no IPv6 CIDR blocks associated with your VPC. |      |                   |        |

Add new IPv6 CIDR

Close

The Secondary IPv4 CIDR blocks associated with my VPC , will now have a route added to their VPC route tables automatically. Adding Secondary CIDR block to existing CIDR block is the go to approach here and it does not lead to EC2 instance downtime because AWS will handle the network configurations and related task for us automatically behind the scenes.

## Q5. Attach two subnets (one public, one private) to the same route table by mistake.

Show:

1. What breaks
2. Which resource becomes exposed or unreachable

Ans :

1. Attaching two subnets i.e private subnet and public subnet to same route table will directly defy the role they play . That is the route table is the only entity that decides what will be working as public subnet and what will be working as the private subnet based on the route table entries . So if both the entries are present in the same route table with conflicting entry for the 0.0.0.0/0 route so either of them will function in same way.

2. If private subnet is added to the route table of public subnet then it will become a public subnet leading to exposing the private databases records or any private application related configuration. Whereas If public subnet is added to the route table of private subnet then it will behave as a private subnet and will not be able to communicate to the internet and will not be able to receive the inbound traffic from the network .

## Q6. Create a setup where:

- NACL allows traffic
  - Security Group blocks traffic
- Then reverse it:

- Security Group allows
- NACL blocks

For each case, prove which layer blocked traffic using logs or behavior (not theory).

Ans :

The screenshot displays the AWS Management Console interface for an EC2 instance named 'piyush-ec2' (Instance ID: i-082019e6fd3fc7f18). The instance is in a 'Running' state and is located in the 'eu-north-1a' Availability Zone. The console shows various details including the instance's IP address (13.48.135.95), hostname, and IAM role. Below the instance details, the 'Security groups' section is expanded, showing a single security group 'sg-02c9313b8ad64f9fd (demo-group)'. The 'Inbound rules' section is also expanded, showing a table with columns: Name, Security group rule ID, Port range, Protocol, Source, Security groups, and Description. The table is currently empty, indicating 'No rules to display'. The 'Outbound rules' section is also expanded, showing a table with columns: Name, Security group rule ID, Port range, Protocol, Destination, Security groups, and Description. The table contains one rule with the following details: Name: -, Security group rule ID: sgr-Od24aac194bc3a031, Port range: All, Protocol: All, Destination: 0.0.0.0/0, Security groups: demo-group, and Description: -.



acl-0754f31fb6d04910f / my-nacl

Details

Inbound rules

Outbound rules

Subnet associations

Tags

Details

Network ACL ID

acl-0754f31fb6d04910f

Associated with

subnet-0f860395700784860 / public-subnet

Default

No

VPC ID

vpc-03fb06f23e8bc4fd8 / piyush-vpc

Owner

797265029671

I have created my setup with my EC2 instance created in my public subnet with my custom Security group attached to which allows outbound traffic but no inbound traffic is allowed in it only ssh traffic is allowed from anywhere i.e 0.0.0.0/0 to check the server. Then in my VPC I have Network ACL which allows all inbound and outbound traffic and my public subnet is associated with it .

So this is the first case where NACL allows traffic but the security group blocks the traffic.

```

Last login: Wed Jan 28 09:18:54 2026 from 110.225.85.240
[ec2-user@ip-10-0-1-110 ~]$ ping google.com
PING google.com (216.58.207.206) 56(84) bytes of data.
^C
--- google.com ping statistics ---
70 packets transmitted, 0 received, 100% packet loss, time 71785ms

[ec2-user@ip-10-0-1-110 ~]$
```

So it is clear that the connection could not reach the EC2 instance because the security group layer did not allow the outbound connection to reach the network .

Now the second case where the NACL blocks all the traffic but the security group allows the traffic.

acl-0754f31fb6d04910f / my-nacl

Details

Inbound rules

Outbound rules

Subnet associations

Tags

Details

Network ACL ID

acl-0754f31fb6d04910f

Associated with

subnet-0f860395700784860 / public-subnet

Default

No

VPC ID

vpc-03fb06f23e8bc4fd8 / piyush-vpc

Owner

797265029671

Inbound rules (2)

Filter inbound rules

| Rule number | Type        | Protocol | Port range | Source    | Allow/Deny |
|-------------|-------------|----------|------------|-----------|------------|
| 100         | All traffic | All      | All        | 0.0.0.0/0 | Deny       |
| *           | All traffic | All      | All        | 0.0.0.0/0 | Deny       |

Edit inbound rules

acl-0754f31fb6d04910f / my-nacl

Details

Inbound rules

Outbound rules

Subnet associations

Tags

Details

Network ACL ID

acl-0754f31fb6d04910f

Associated with

subnet-0f860395700784860 / public-subnet

Default

No

VPC ID

vpc-03fb06f23e8bc4fd8 / piyush-vpc

Owner

797265029671

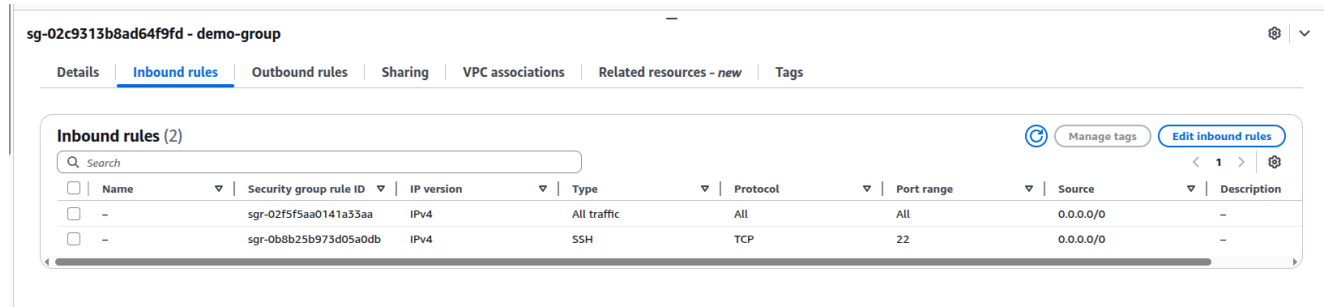
Outbound rules (1)

Filter outbound rules

| Rule number | Type        | Protocol | Port range | Destination | Allow/Deny |
|-------------|-------------|----------|------------|-------------|------------|
| *           | All traffic | All      | All        | 0.0.0.0/0   | Deny       |

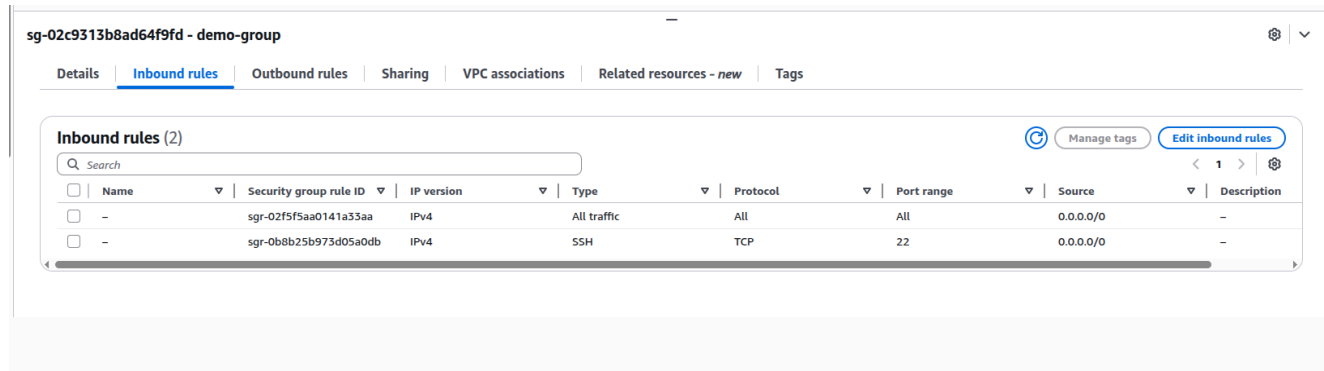
Edit outbound rules

The above snapshots shows the NACL rules changed.

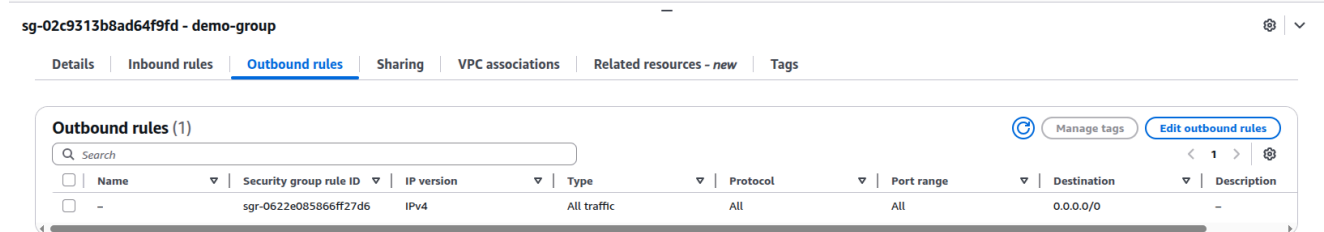


| Name | Security group rule ID | IP version | Type        | Protocol | Port range | Source    | Description |
|------|------------------------|------------|-------------|----------|------------|-----------|-------------|
| -    | sgr-02f5f5aa0141a33aa  | IPv4       | All traffic | All      | All        | 0.0.0.0/0 | -           |
| -    | sgr-0b8b25b973d05a0db  | IPv4       | SSH         | TCP      | 22         | 0.0.0.0/0 | -           |

Now the following snapshots shows the security groups rules to be changed.



| Name | Security group rule ID | IP version | Type        | Protocol | Port range | Source    | Description |
|------|------------------------|------------|-------------|----------|------------|-----------|-------------|
| -    | sgr-02f5f5aa0141a33aa  | IPv4       | All traffic | All      | All        | 0.0.0.0/0 | -           |
| -    | sgr-0b8b25b973d05a0db  | IPv4       | SSH         | TCP      | 22         | 0.0.0.0/0 | -           |



| Name | Security group rule ID | IP version | Type        | Protocol | Port range | Destination | Description |
|------|------------------------|------------|-------------|----------|------------|-------------|-------------|
| -    | sgr-0622e085866ff27d6  | IPv4       | All traffic | All      | All        | 0.0.0.0/0   | -           |

Now when I block the traffic from NACL .The NACL layer blocks the traffic and does not even allow only to go to the Security groups to verify the rules. So the connection is timed out and it fails.

```
^C
[ec2-user@ip-10-0-1-110 ~]$ ping google.com
PING google.com (142.250.74.110) 56(84) bytes of data.
^C
--- google.com ping statistics ---
21 packets transmitted, 0 received, 100% packet loss, time 20818ms

[ec2-user@ip-10-0-1-110 ~]$
```

## Q7. Create three VPCs (A, B, C).

Peer A ↔ B and B ↔ C.

Prove (with testing) that A cannot reach C.

Explain why AWS enforces this limitation and one valid architectural alternative.

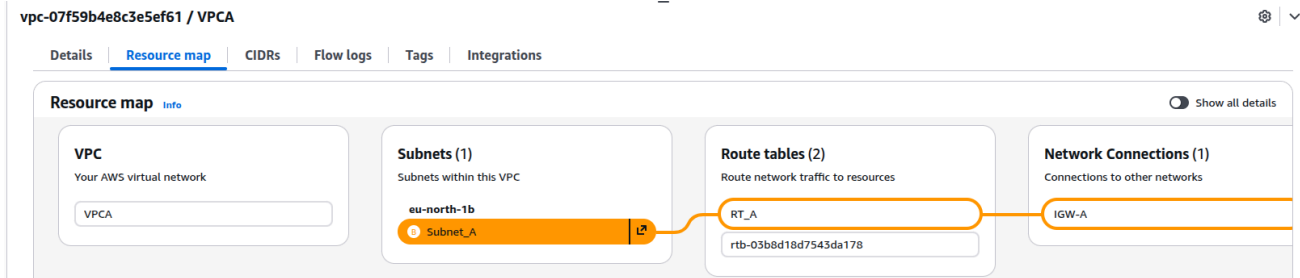
**\*\*Ans :**

**Peering connections (1/2)** Info

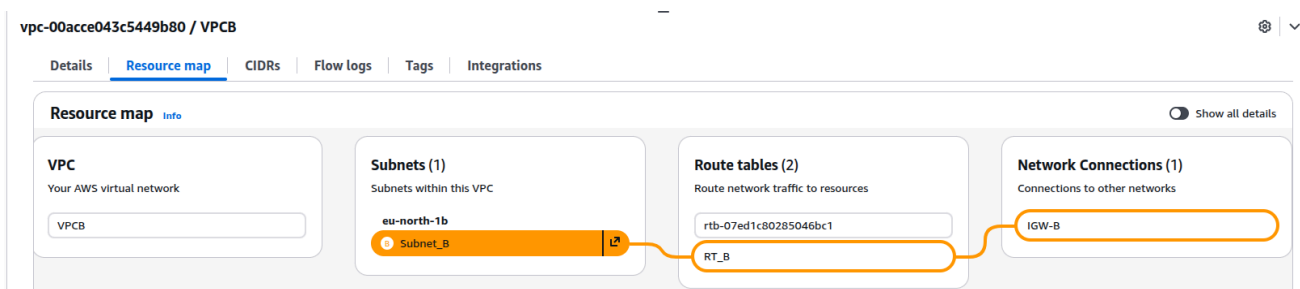
Find peering connections by attribute or tag

| Name       | Peering connection ID | Status | Requester VPC                | Accepter VPC                 | Requester CIDRs | Accepter CIDRs | Req |
|------------|-----------------------|--------|------------------------------|------------------------------|-----------------|----------------|-----|
| peering-AB | pcx-0a6ece569a7c72b5e | Active | vpc-07f59b4e8c3e5ef61 / VPCA | vpc-00acce043c5449b80 / VPCB | 192.168.0.0/24  | 192.168.1.0/24 | 79% |
| peeringBC  | pcx-0eb67d1ca853b9600 | Active | vpc-00acce043c5449b80 / VPCB | vpc-0332dbf36355c2bfd / VPCC | 192.168.1.0/24  | 192.168.2.0/24 | 79% |

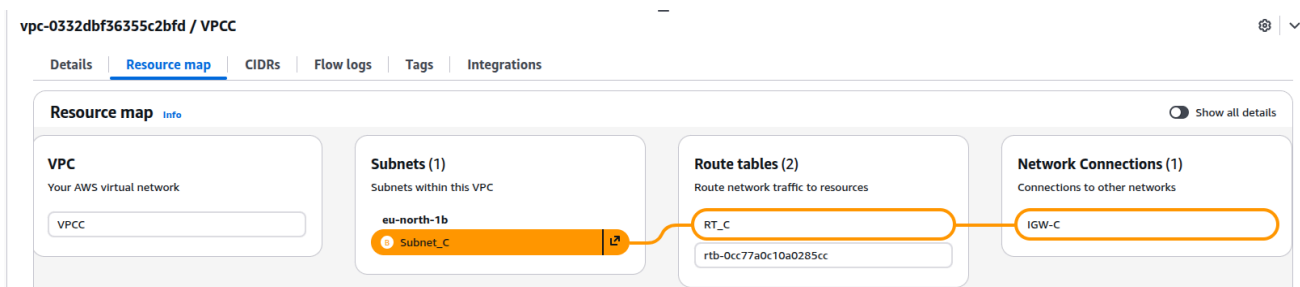
My three VPCs are connected and I have peered A-B and B-C .



This is VPC - A



This is VPC -B



This is VPC-C

I have my three instances running in each VPC to check how the peering will work each instance in a separate VPC and each VPC has a public subnet and associated internet gateway to it.

**Instances (4)** Info

Find Instance by attribute or tag (case-sensitive)

All states

| Name  | Instance ID         | Instance state | Instance type | Status check      | Alarm status  | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elastic II |
|-------|---------------------|----------------|---------------|-------------------|---------------|-------------------|-----------------|-----------------|------------|
| Cec2  | i-0727943e801003cf2 | Running        | t3.micro      | 3/3 checks passed | View alarms + | eu-north-1b       | -               | 13.48.30.5      | -          |
| A-ec2 | i-0491feb92f1c11a2c | Running        | t3.micro      | 3/3 checks passed | View alarms + | eu-north-1b       | -               | 13.60.55.235    | -          |
| B-ec2 | i-0519f7b10bcdf38a3 | Running        | t3.micro      | 3/3 checks passed | View alarms + | eu-north-1b       | -               | 13.60.30.113    | -          |

Select an instance

First Checking the VPC peering from A to B by first login into EC2 instance A and then using private ip of instance B the connection should succeed as shown because of peering and

route between A and B.

```
Last login: Wed Jan 28 15:11:05 2026 from 103.83.148.66
[ec2-user@ip-192-168-0-12 ~]$ ping 192.168.1.28
PING 192.168.1.28 (192.168.1.28) 56(84) bytes of data.
64 bytes from 192.168.1.28: icmp_seq=1 ttl=127 time=0.181 ms
64 bytes from 192.168.1.28: icmp_seq=2 ttl=127 time=0.184 ms
64 bytes from 192.168.1.28: icmp_seq=3 ttl=127 time=0.185 ms
64 bytes from 192.168.1.28: icmp_seq=4 ttl=127 time=0.166 ms
64 bytes from 192.168.1.28: icmp_seq=5 ttl=127 time=0.195 ms
64 bytes from 192.168.1.28: icmp_seq=6 ttl=127 time=0.185 ms
```

Now we see the connection between A and C , the ping to C's private network does not succeed showing that peering between A and C is not possible through B.

```
[ec2-user@ip-192-168-0-12 ~]$ ping -c 5 192.168.2.53
PING 192.168.2.53 (192.168.2.53) 56(84) bytes of data.
--- 192.168.2.53 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4133ms

[ec2-user@ip-192-168-0-12 ~]$
```

Now I login in instance C , i.e VPC C first I ping Ec2 in VPC B using its private IP , the connection succeeds

```
[ec2-user@ip-192-168-2-53 ~]$ ping 192.168.1.28
PING 192.168.1.28 (192.168.1.28) 56(84) bytes of data.
64 bytes from 192.168.1.28: icmp_seq=1 ttl=127 time=0.419 ms
64 bytes from 192.168.1.28: icmp_seq=2 ttl=127 time=0.172 ms
64 bytes from 192.168.1.28: icmp_seq=3 ttl=127 time=0.202 ms
```

Now I check for VPC A ,

```
ec2-user@ip-192-168-2-53:~
[ec2-user@ip-192-168-2-53 ~]$ ping -c 5 192.168.0.12
PING 192.168.0.12 (192.168.0.12) 56(84) bytes of data.
--- 192.168.0.12 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4174ms

[ec2-user@ip-192-168-2-53 ~]$
```

The connection fails.

The best and alternative approach for this would be to use transit gateway which supports transitive routing , i.e VPC-A -> TGW -> VPC-C and it can be scaled to hundreds of VPCs.

## Q8. Peer two VPCs in different regions.

Run a basic latency test (ping / traceroute / app response).

Record:

1. Observed latency
2. Why latency exists even though traffic is “private”

Ans :

This is my Region One in EU- Stockholm which initiates the VPC peering request.

The screenshot shows the AWS VPC console for a peering connection named **pcx-0a4d127c1363758a3 / EU-US-peering**. A green banner at the top states: "A VPC peering connection pcx-0a4d127c1363758a3 / EU-US-peering has been requested. Remember to change your region to us-east-1 to accept the peering connection." The connection is in the **Initiating** state. The requester VPC is **vpc-0d49d7ad119f92ee4** in the **Stockholm (eu-north-1)** region. The accepter VPC is **vpc-09d024373d1186fc3** in the **N. Virginia (us-east-1)** region. The status is "Initiating Request to 797265029671". The expiration time is "Wednesday, February 4, 2026 at 22:02:52 GMT+5:30". The DNS settings are currently disabled for both directions.

| Details               | Requester VPC                                    | Accepter VPC                                     |
|-----------------------|--------------------------------------------------|--------------------------------------------------|
| Requester owner ID    | 797265029671                                     | 797265029671                                     |
| Peering connection ID | pcx-0a4d127c1363758a3                            | pcx-0a4d127c1363758a3                            |
| Status                | Initiating                                       | Initiating                                       |
| Expiration time       | Wednesday, February 4, 2026 at 22:02:52 GMT+5:30 | Wednesday, February 4, 2026 at 22:02:52 GMT+5:30 |
| Requester Region      | Stockholm (eu-north-1)                           | N. Virginia (us-east-1)                          |
| Requester CIDRs       | 10.0.0.0/24                                      | 10.0.0.0/24                                      |
| Accepter CIDRs        | 10.0.0.0/24                                      | 10.0.0.0/24                                      |

The Acceptor is in Region US-N.Virginia which accepts the request and the peering is active and successful.

The screenshot shows the same VPC peering connection, but now it is in the **Active** state. The status is "Active". The requester VPC is **vpc-0d49d7ad119f92ee4** in the **Stockholm (eu-north-1)** region. The accepter VPC is **vpc-0b8d66de1160566a5** in the **N. Virginia (us-east-1)** region. The expiration time is "—". The DNS settings are still disabled.

| Details               | Requester VPC          | Accepter VPC            |
|-----------------------|------------------------|-------------------------|
| Requester owner ID    | 797265029671           | 797265029671            |
| Peering connection ID | pcx-0a1cdbcdfdeb9ff02  | pcx-0a1cdbcdfdeb9ff02   |
| Status                | Active                 | Active                  |
| Expiration time       | —                      | —                       |
| Requester Region      | Stockholm (eu-north-1) | N. Virginia (us-east-1) |
| Requester CIDRs       | 10.0.0.0/24            | 10.0.0.0/24             |
| Accepter CIDRs        | 10.0.0.0/24            | 10.0.0.0/24             |

1. Now run basic test to check the latency , I have deployed two EC2 instances one in each region . So I login in EC2 instance at EU-Stockholm and then ping at EC2 in US region .

```

64 bytes from 10.1.0.5: icmp_seq=24 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=25 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=26 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=27 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=28 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=29 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=30 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=31 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=32 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=33 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=34 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=35 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=36 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=37 ttl=127 time=111 ms
64 bytes from 10.1.0.5: icmp_seq=38 ttl=127 time=111 ms
^C
--- 10.1.0.5 ping statistics ---
38 packets transmitted, 38 received, 0% packet loss, time 37054ms
rtt min/avg/max/mdev = 111.338/111.374/111.445/0.030 ms
[ec2-user@ip-10-0-0-8 ~]$

```

Using ping the observed late avg =111.338 which is very poor and slower when compared with VPC peering in same region .

```

[ec2-user@ip-10-0-0-8 ~]$ traceroute 10.1.0.5
traceroute to 10.1.0.5 (10.1.0.5), 30 hops max, 60 byte packets
 1 ip-10-1-0-5.eu-north-1.compute.internal (10.1.0.5) 110.303 ms * *
[ec2-user@ip-10-0-0-8 ~]$

```

Using traceroute as well the result were same only .

2. The latency exists even though the traffic uses private IPs because traffic is in reality existing on AWS Global Network which already very congested . Moreover the regions being far away one in US and one in EU the data takes time to reach from one server to another via VPC peering travelling around multiple locations.

## Q9. Enable VPC Flow Logs at ENI level.

Generate:

- One ACCEPT log
- One REJECT log

Explain:

1. Which configuration caused each
2. How Flow Logs helped you identify the issue

**Ans:**

I created Flow Log at ENI level for my VPC named My-Flow-Log

Successfully created flow log for the following resource:  
vpc-0ed8ba79554436448

fl-0bfaaf21190180e62 / My-Flow-Log

Actions

Details

Flow Log ID  
fl-0bfaaf21190180e62

Name  
My-Flow-Log

State  
Active

Creation Time  
Wednesday, January 28, 2026 at 22:43:43 GMT+5:30

Destination Type  
cloud-watch-logs

Destination Name  
aws/vpc/eni-flowlogs

IAM Role  
arn:aws:iam::797265029671:role/service-role/VPCFlowLogs-Cloudwatch-1769620316108

Cross Account IAM Role  
-

Traffic Type  
All

Max Aggregation Interval  
1 minute

Log Format  
Default

File Format  
-

Hive Compatible Partitions  
-

Partition Logs  
-

Tags

Integrations

Tags

Search tags

Manage tags

Key

Value

Name

My-Flow-Log

Now generating one accept flow log by logging in to my EC2 instance using public IP.

Log events

You can use the filter bar below to search for and match terms, phrases, or values in your log events. [Learn more about filter patterns](#)

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom UTC timezone Display

| Timestamp                | Message                                                                                                          |
|--------------------------|------------------------------------------------------------------------------------------------------------------|
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 35.203.211.139 45101 523/0 6 1 40 1769620961 1769620987 ACCEPT OK |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 35.176.149.124 10.0.0.22 123 55374 17 1 76 1769620961 1769620987 ACCEPT OK  |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 35.176.149.124 55374 123 17 1 76 1769620961 1769620987 ACCEPT OK  |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 121.52.153.7 10.0.0.22 2634 445 6 1 52 1769620961 1769620987 ACCEPT OK      |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 121.52.153.7 445 2634 6 1 40 1769620961 1769620987 ACCEPT OK      |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 147.185.133.75 10.0.0.22 53286 45063 6 1 44 1769620961 1769620987 ACCEPT OK |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 147.185.133.75 45063 53286 6 1 40 1769620961 1769620987 ACCEPT OK |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 109.205.211.80 10.0.0.22 44325 57665 6 1 40 1769620961 1769620987 ACCEPT OK |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 109.205.211.80 57665 44325 6 1 40 1769620961 1769620987 ACCEPT OK |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 91.196.152.213 10.0.0.22 42803 5556 6 1 60 1769620961 1769620987 ACCEPT OK  |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 91.196.152.213 5556 42803 6 1 40 1769620961 1769620987 ACCEPT OK  |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 66.92.207.178 10.0.0.22 33444 80 6 1 81 1769620961 1769620987 ACCEPT OK     |
| 2026-01-28T17:22:41.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 66.92.207.178 80 33444 6 1 40 1769620961 1769620987 ACCEPT OK     |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 13.40.171.61 10.0.0.22 123 47568 17 1 76 1769620995 1769621013 ACCEPT OK    |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 13.40.171.61 47568 123 17 1 76 1769620995 1769621013 ACCEPT OK    |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 35.203.211.203 10.0.0.22 52916 9037 6 1 44 1769620995 1769621013 ACCEPT OK  |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 35.203.211.203 9037 52916 6 1 40 1769620995 1769621013 ACCEPT OK  |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 18.134.134.61 10.0.0.22 123 47093 17 1 76 1769620995 1769621013 ACCEPT OK   |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 10.0.0.22 18.134.134.61 47093 123 17 1 76 1769620995 1769621013 ACCEPT OK   |
| 2026-01-28T17:23:15.000Z | 2 797265029671 eni-099d44d3265098d09 167.94.138.136 10.0.0.22 19081 18082 6 1 60 1769620995 1769621013 ACCEPT OK |

My flow logs shows the Accpeted log entry in the cloudwatch logs section.

Now generating one reject flow log by changing the inbound rules to now allow any traffic from outside and trying to connect to another ec2 instance .

2026-01-28T22:58:55.000+05:30 2 797265029671 eni-0c703d17f34be29a7 193.163.125.100 10.0.0.59 37481 30077 6 1 44 1769621335 1769621360 REJECT OK

2026-01-28T22:58:55.000+05:30 2 797265029671 eni-0c703d17f34be29a7 162.216.150.67 10.0.0.59 53343 2012 6 1 44 1769621335 1769621360 REJECT OK

- The accept log was generated because the inbound rule specified in security group created by me allowed the traffic from anywhere so the accept log was generated . The reject log was generated because after removing the inbound rule which allowed traffic from anywhere the ec2 instance was not able to reach the requesting instance . Hence generating the reject log.
- The flow logs clearly showed the requesting IP and the request being accepted or rejected along with the IP of sender and receiver in the standard format showing interface id where traffic is evaluated , source IP , destination IP along with port details, so we can track and find where the network connectivity issue lies.

## Q10. Using only NACLs, block a single public IP from accessing an EC2 instance while allowing another IP.

Prove results with:

1. Rule number priority
2. Connection attempts
3. Outcome screenshots/logs

Ans :

acl-08d0e3c8b402dec10 / NACL-10

Details | **Inbound rules** | Outbound rules | Subnet associations | Tags

### Inbound rules (5)

Filter inbound rules

Edit inbound rules

| Rule number | Type            | Protocol | Port range | Source          | Allow/Deny |
|-------------|-----------------|----------|------------|-----------------|------------|
| 100         | SSH (22)        | TCP (6)  | 22         | 13.16.17.226/32 | Deny       |
| 110         | SSH (22)        | TCP (6)  | 22         | 13.49.70.215/32 | Allow      |
| 120         | All ICMP - IPv4 | ICMP (1) | All        | 0.0.0.0/0       | Allow      |
| 140         | All traffic     | All      | All        | 0.0.0.0/0       | Deny       |
| *           | All traffic     | All      | All        | 0.0.0.0/0       | Deny       |

I have created two EC2 instances in a VPC and in same public subnets attached to same security groups . Now I have configure NACL in a way to block one Ec2 instance and allow another by configuring the inbound and outbound rules.

acl-08d0e3c8b402dec10 / NACL-10

Details | Inbound rules | **Outbound rules** | Subnet associations | Tags

### Outbound rules (4)

Filter outbound rules

Edit outbound rules

| Rule number | Type            | Protocol | Port range   | Destination | Allow/Deny |
|-------------|-----------------|----------|--------------|-------------|------------|
| 100         | Custom TCP      | TCP (6)  | 1024 - 65535 | 0.0.0.0/0   | Allow      |
| 110         | All ICMP - IPv4 | ICMP (1) | All          | 0.0.0.0/0   | Allow      |
| 140         | All traffic     | All      | All          | 0.0.0.0/0   | Deny       |
| *           | All traffic     | All      | All          | 0.0.0.0/0   | Deny       |

1. The NACL will resolve the requests based on the above rules , based on the priority i.e the smaller the number will be evaluated first .

The allowed EC2 is : 13.49.70.215/32 and The blocked EC2 is : 13.16.17.226/32

Now when I used the ping on the blocked EC2 ip the packets were dropped and



connection were successful , this shows the NACL blocking the EC2 instance.

```
piyush@CK-piyush:~$ ping 13.61.17.226
PING 13.61.17.226 (13.61.17.226) 56(84) bytes of data.
^C
--- 13.61.17.226 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1052ms

piyush@CK-piyush:~$
```

Now the allowed ec2 instance is able to capture packet and connect successfully because the nacl decided the traffic by the rule priority.

```
piyush@CK-piyush:~$ ping 13.49.70.215
PING 13.49.70.215 (13.49.70.215) 56(84) bytes of data.
64 bytes from 13.49.70.215: icmp_seq=1 ttl=107 time=323 ms
64 bytes from 13.49.70.215: icmp_seq=2 ttl=107 time=347 ms
64 bytes from 13.49.70.215: icmp_seq=3 ttl=107 time=371 ms
64 bytes from 13.49.70.215: icmp_seq=4 ttl=107 time=293 ms
```

### Q11. Configure an EC2 instance that:

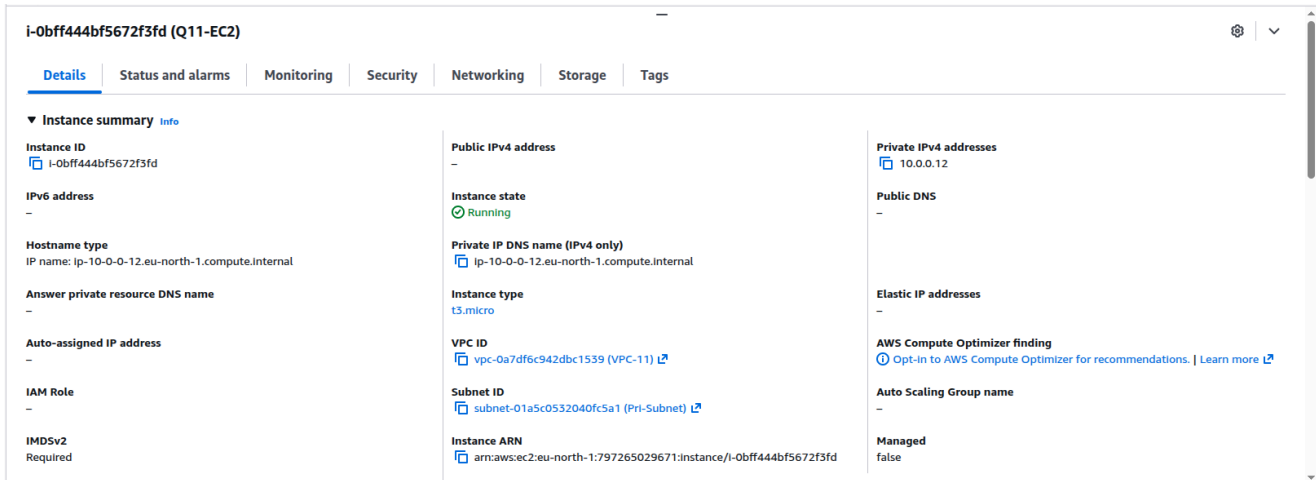
- Has no IGW
- Has no NAT Gateway
- Still accesses S3 successfully

Explain:

1. Exact service used
2. Route table changes
3. Why traffic never touches the public internet

**Ans:**

I have configured the EC2 instance with no IGW and NAT gateway attached to it.

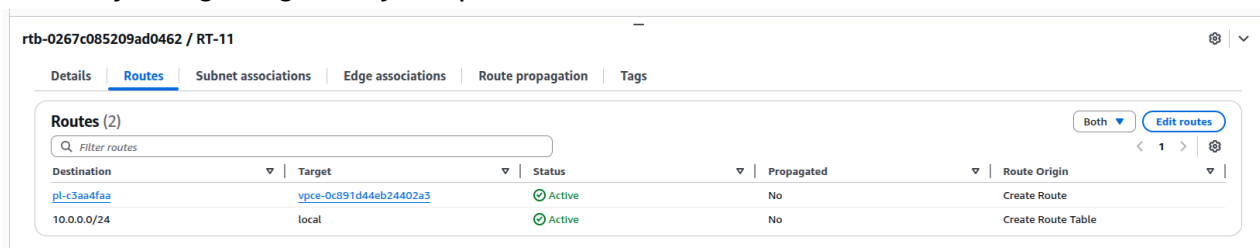


```
piyush@CK-piyush:~$ ping 56.228.33.213
PING 56.228.33.213 (56.228.33.213) 56(84) bytes of data.
^C
--- 56.228.33.213 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3032ms

piyush@CK-piyush:~$
```

This shows that the Ec2 instance configured has no internet access to it.

1. So now to access the S3 service without IGW and NAT gateway I created a VPC endpoint gateway which allows to communicate within the AWS local network without needing the internet connection . The S3 service uses mainly VPC gateway endpoint so I choose these.
2. When VPC gateway endpoint is created and attached to my vpc , AWS automatically configures the route table for us and create the route table entry to route the traffic internally using the gateway endpoint service.



3. The traffic never touches the public internet because we don't have IGW connected to our subnet or no 0.0.0.0/0 CIDR entry into our route table , so the traffic stays within the AWS private network or internal network and never reach public internet.

## Q12. Given a private application that accesses S3 and CloudWatch, decide:

- A. Which endpoint type is required for each
  - B. Why one cannot be used for both
- Back your answer with actual configuration proof.

Ans :

A. The Ec2 Configuration with no IGW and not NAT , so here again for S3 we will use Gateway endpoint and for Cloudwatch we will use Interface endpoint because it provided private access to AWS Services and our own services over a private link.

i-00c3e1b2fa2b57464 (Q12Ec2)

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

▼ Instance summary

Info

Instance ID

i-00c3e1b2fa2b57464

IPv6 address

—

Hostname type

IP name: ip-10-0-0-37.eu-north-1.compute.internal

Answer private resource DNS name

—

Auto-assigned IP address

13.50.249.255 [Public IP]

IAM Role

—

IMDSv2

Required

Public IPv4 address

13.50.249.255 | [open address](#)

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-0-0-37.eu-north-1.compute.internal

Instance type

t3.micro

VPC ID

vpc-099228703e775a841 (Q12) [↗](#)

Subnet ID

subnet-080719a701b8c7b52 (Q12PrivateSubnet) [↗](#)

Instance ARN

arn:aws:ec2:eu-north-1:797265029671:instance/i-00c3e1b2fa2b57464

Private IPv4 addresses

10.0.0.37

Public DNS

—

Elastic IP addresses

—

AWS Compute Optimizer finding

[Opt-in to AWS Compute Optimizer for recommendations.](#) | [Learn more](#) [↗](#)

Auto Scaling Group name

—

Managed

false

B. One thing cannot be used for both services because gateway endpoint maintains private prefix list of IPs to enable secure and direct routing from VPC to the services without traversing public internet and Cloudwatch do not have any prefix list .

| rtb-0918d78ec4f1474f8 / RT12                         |                                        |                     |                   |                    |      |
|------------------------------------------------------|----------------------------------------|---------------------|-------------------|--------------------|------|
| Details                                              | Routes                                 | Subnet associations | Edge associations | Route propagation  | Tags |
| Routes (2)                                           |                                        |                     |                   |                    |      |
| <div>Filter routes</div> <div>Both Edit routes</div> |                                        |                     |                   |                    |      |
| Destination                                          | Target                                 | Status              | Propagated        | Route Origin       |      |
| <a href="#">pl-c3aa4faa</a>                          | <a href="#">vpce-0cabf83dd10ce3992</a> | Active              | No                | Create Route       |      |
| 10.0.0.0/24                                          | local                                  | Active              | No                | Create Route Table |      |

The route table entry changed as soon as we created the VPC gateway endpoint for S3 service.

| Network interface: eni-072d6ac47460a2e68                      |                                                       |                                                                              |
|---------------------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------|
| Details                                                       | Flow logs                                             | Tags                                                                         |
| ▼ Network interface details                                   |                                                       |                                                                              |
| Network interface ID<br><a href="#">eni-072d6ac47460a2e68</a> | Name<br>-                                             | Description<br><a href="#">VPC Endpoint Interface vpce-09281e7456331f76a</a> |
| Network interface status<br><span>In-use</span>               | Interface type<br><a href="#">vpc_endpoint</a>        | Security groups<br><a href="#">sg-01529d4fac2c2d895</a> (Q125G)              |
| VPC ID<br><a href="#">vpc-099228703e775a841</a>               | Subnet ID<br><a href="#">subnet-080719a701b8c7b52</a> | Availability Zone<br><a href="#">eu-north-1c</a>                             |
| Owner<br><a href="#">797265029671</a>                         | Requester ID<br><a href="#">727180483921</a>          | Requester-managed<br>True                                                    |
| Source/dest. check<br>True                                    | Managed<br>False                                      | Operator<br>-                                                                |
| ▼ IP addresses                                                |                                                       |                                                                              |
| Private IPv4 address<br><a href="#">10.0.0.59</a>             | Elastic Fabric Adapter<br>False                       | Public IPv4 address<br>-                                                     |
| IPv6 addresses<br>-                                           | Secondary public IPv4 addresses<br>-                  | Secondary private IPv4 addresses<br>-                                        |

The ENI was automatically created for the EC2 instance as soon as the endpoint was created.

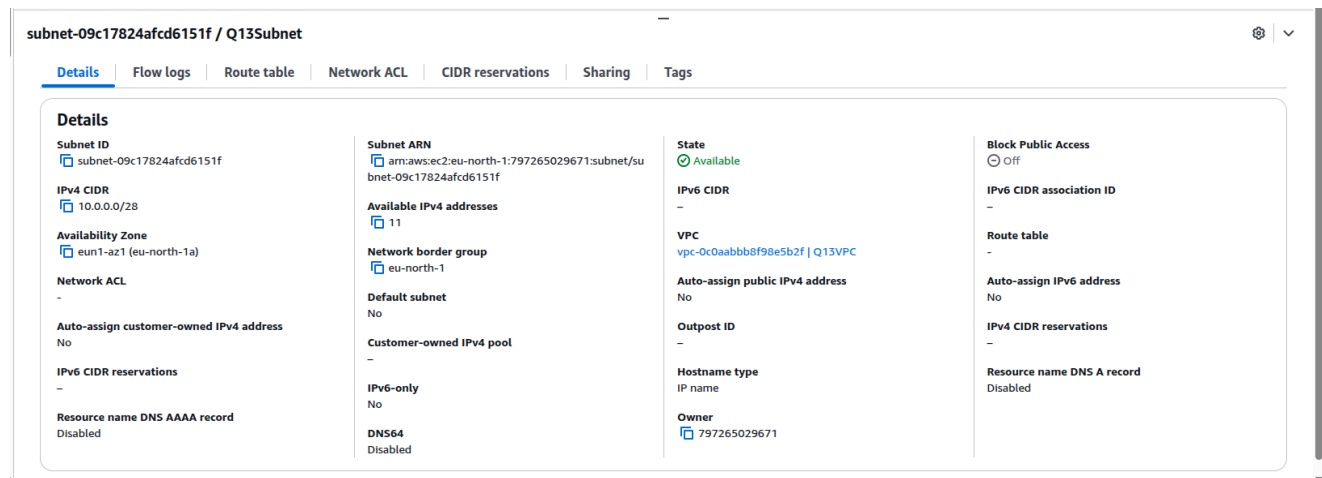
## Q13. Launch a subnet with a very small CIDR (e.g., /28).

List:

- All reserved IPs  
Which ones AWS uses and why  
Show what happens if you try to assign them manually.

Ans :

The subnet with CIDR launched inside the VPC with CIDR as : 10.0.0.0/24



The subnet will have IPs from the range :  $32 - 28 = 4 = 2^4 = 16$

Full IP address range will be : 10.0.0.0 - 10.0.0.15

AWS will reserves 5 IPs from these IPs .

The reserved IPs will be :

10.0.0.0 => Network Address

10.0.0.1 => VPC Router

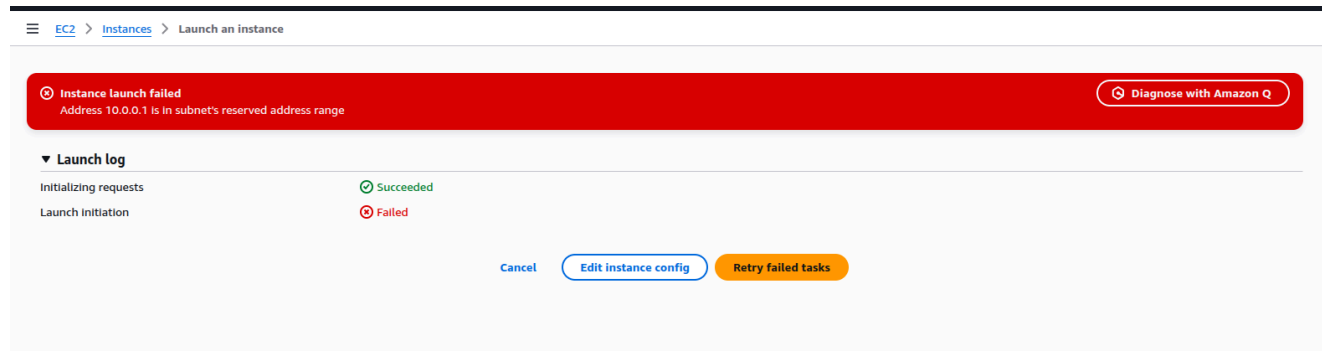
10.0.0.2 => AWS DNS

10.0.0.3 => Future use

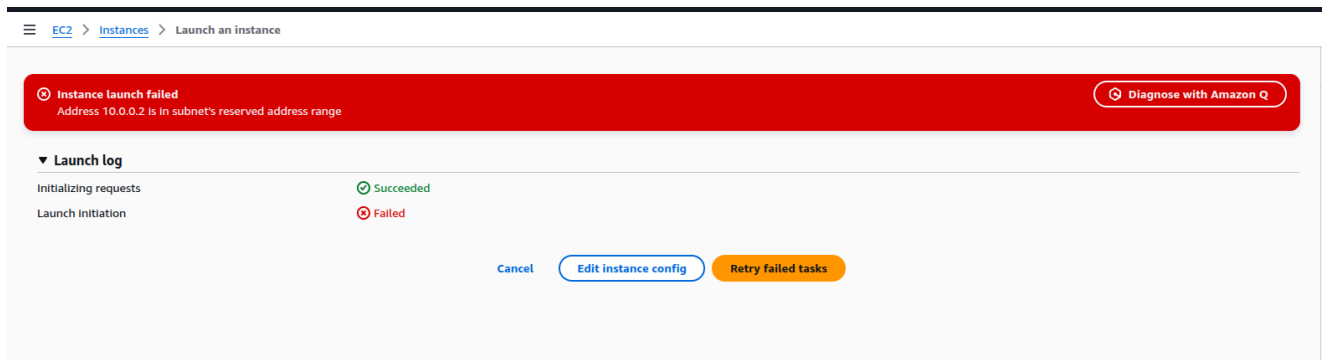
10.0.0.15 => Broadcast- like address used by AWS

So the total remaining IPs which are usable lies in the range : 10.0.0.4 - 10.0.0.14

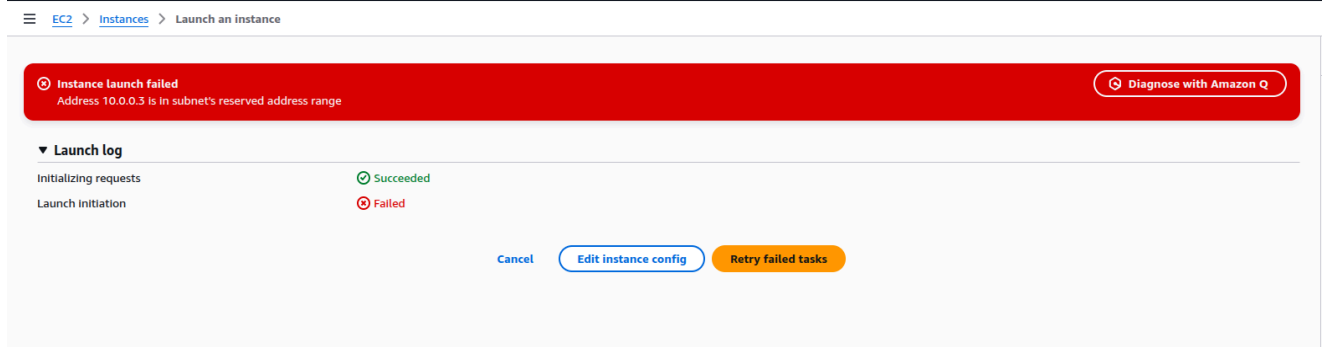
When tried assigning the reserved IP : 10.0.0.1 manually the AWS throws an error.



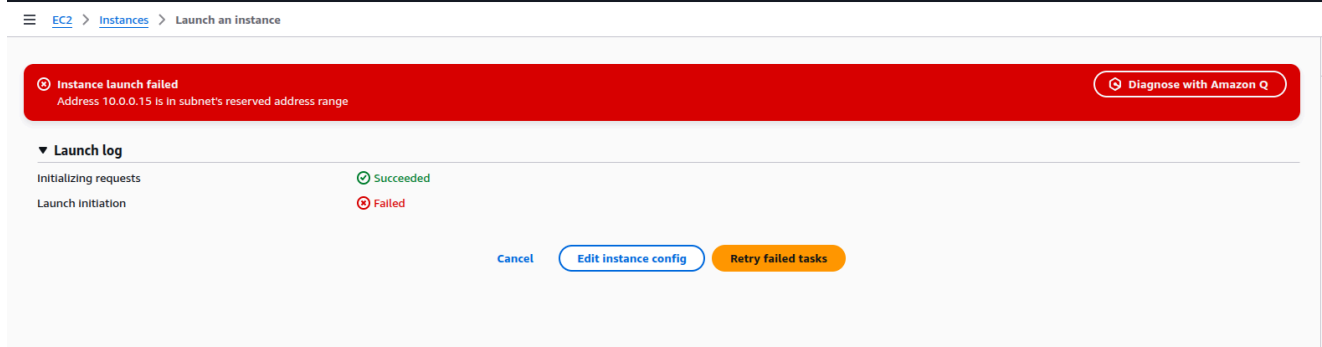
For 10.0.0.2 it throws



For 10.0.0.3 it throws error



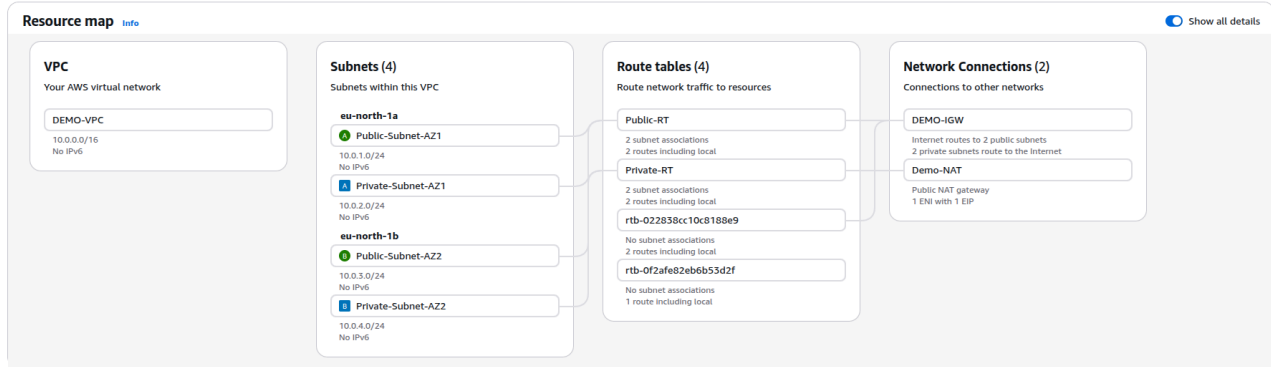
For 10.0.0.15 it throws error



**Q14. Create a clean architecture diagram (using draw.io) of your final VPC showing:**

- CIDRs
  - AZs
  - Route tables
  - Gateways
  - Security layers
- Rules:
- No default VPC components
  - Every arrow must represent real traffic
  - Diagram must match your AWS console exactly

**Ans:**



Now mirroring the same clean architecture using draw.io

